Proof. We prove the lemma by contradiction. Assume $w' \prec w$. Then by the definition of ordered trees there are $u, p, v, v'$ such that $w' = u.p.v'$ and $w = u.v$ such that $u.p \prec u.v$. Since $u.v$ is a leaf, we have $p \notin WI_{[E.u]}(v)$ by Property 2 of Definition 6.1. Hence $p \not\prec_{[E.u]} v$, which implies $u.p \not\prec_{[E]} u.v$, which implies $u.p.v' \not\prec_{[E]} u.v$, i.e., $w' \not\prec_{[E]} w$. □

For a wakeup tree $\langle B, \prec \rangle$ and a process $p \in B$, define $subtree(\langle B, \prec \rangle, p)$ to denote the subtree of $\langle B, \prec \rangle$ rooted at $p$, i.e., $subtree(\langle B, \prec \rangle, p) = \langle B', \prec' \rangle$ where $B' = \{w \mid p.w \in B\}$ and $\prec'$ is the restriction of $\prec$ to $B'$.

### 6.2 Inserting Sequences in a Wakeup Tree

Let $\langle B, \prec \rangle$ be a wakeup tree after $\langle E, P \rangle$. For any sequence $w$ such that $E.w$ is an execution sequence with $WI_{[E]}(w) \cap P = \emptyset$, we need an operation $insert_{[E]}(w, \langle B, \prec \rangle)$, that satisfies the following properties:

(1) $insert_{[E]}(w, \langle B, \prec \rangle)$ is also a wakeup tree after $\langle E, P \rangle$,
(2) any leaf of $\langle B, \prec \rangle$ remains a leaf of $insert_{[E]}(w, \langle B, \prec \rangle)$, and
(3) $insert_{[E]}(w, \langle B, \prec \rangle)$ contains a leaf $u$ with $u \sim_{[E]} w$.

A simple implementation of $insert_{[E]}(w, \langle B, \prec \rangle)$ is the following: Let $v$ be the smallest (w.r.t. to $\prec$) sequence in $B$ such that $v \sim_{[E]} w$. If $v$ is a leaf, the result of $insert_{[E]}(w, \langle B, \prec \rangle)$ can be taken as $\langle B, \prec \rangle$ (leaving the tree unmodified). Otherwise, let $w'$ be a shortest sequence such that $w \sqsubseteq_{[E]} v.w'$, and add $v.w'$ as a new leaf, which is ordered after all already existing nodes in $B$ of form $v.w''$.

As an illustration, using the program of Fig. 1, assume that a wakeup tree $\langle B, \prec \rangle$ after $\langle \langle \rangle, \emptyset \rangle$ contains $p$ as the only leaf. Then the operation $insert_{[\langle \rangle]}(q.q, \langle B, \prec \rangle)$ adds $q.q$ as a new leaf with $p \prec q.q$. If we thereafter perform $insert_{[\langle \rangle]}(r.r, \langle B, \prec \rangle)$, then the wakeup tree remains the same, since $q.q \sim_{[\langle \rangle]} r.r$, and $q.q$ is already a leaf.

## 7 OPTIMAL-DPOR

In this section, we present a DPOR algorithm that achieves optimal reduction.

### 7.1 Algorithm

The *optimal-DPOR* algorithm, shown as Algorithm 2, performs a depth-first search using the recursive procedure *Explore*($E$, *Sleep*, *WuT*), where $E$ and *Sleep* are as in Algorithm 1, and *WuT* is a wakeup tree after $\langle E, Sleep \rangle$, containing extensions of $E$ that are guaranteed to be explored (in order) by *Explore*($E$, *Sleep*, *WuT*). If *WuT* is empty, then *Explore*($E$, *Sleep*, *WuT*) is free to explore any extension of $E$.

Like Algorithm 1, the algorithm runs in two phases: race detection (lines 2–7) and state exploration (lines 8–21), but it is slightly differently organized. Instead of analyzing races at every invocation of *Explore*, races are analyzed in the entire execution sequence only when a maximal execution sequence has been generated. The reason for this is that the test at line 6 is precise only when the used sequence $v$, which is defined at line 5, includes all events in the entire execution that do not "happen after" $e$, also those that occur after $e'$. Therefore $v$ can be defined only when $E$ is a maximal execution sequence.

In the race detection phase, Algorithm 2 must be able to access the current sleep set for each prefix $E'$ of the currently explored execution sequence $E$. For each such prefix $E'$, the algorithm therefore maintains a set of processes $sleep(E')$, which is the current sleep set at $E'$. In a similar way, for each prefix $E'$ of $E$, the algorithm maintains $wut(E')$, which is the current wakeup tree at $E'$.

Let us now explain the race detection phase, which is entered whenever the exploration reaches the end of a complete sequence (i.e., $enabled(s_{[E]}) = \emptyset$). In this phase, the algorithm investigates all